Laboratorio 3:

Unidad Curricular: Tecnologías de Microprocesamiento
Universidad Tecnológica (UTEC) - Ingeniería Mecatrónica

Tizziano Travers (C.I.: 5.623.448-1)
Bruno Bentancourt (C.I.: 5.495.926-3)
Giuliano Gilardoni (C.I.: 5.644.255-9)
Docentes: Jesús Martínez / Marcelo Díaz

I. Introducción

La transición de la programación en lenguaje ensamblador hacia el lenguaje C representa un hito fundamental en el desarrollo de sistemas embebidos, permitiendo abstraer la complejidad del hardware sin perder el control directo sobre los registros de memoria de la arquitectura AVR. El microcontrolador ATmega328P de 8 bits destaca por su estructura RISC eficiente y una suite versátil de periféricos integrados, lo que facilita el diseño de soluciones de control en tiempo real, procesamiento analógico y generación de señales temporizadas.

En el presente informe se aborda la integración modular de periféricos internos —tales como el convertidor analógico-digital (ADC), temporizadores hardware (Timers) para modulación por ancho de pulso (PWM) y modo CTC, el canal serie de comunicación asíncrona (USART) e interrupciones— combinados con protocolos de comunicación estándar como I2C/TWI y Monobus (1-Wire). A través de esta plataforma, se diseñan e implementan tres sistemas embebidos independientes que resuelven problemas de regulación térmica adaptativa, posicionamiento mecatrónico por realimentación lumínica y síntesis de audio polifónico.

II. Objetivos

II-A. Objetivo General

Diseñar, programar y validar experimentalmente sistemas embebidos de control, adquisición de datos e interfaz de usuario utilizando lenguaje C sobre el microcontrolador ATmega328P, integrando periféricos internos de la arquitectura AVR con sensores, actuadores y herramientas de telemetría en tiempo real.

II-B. Objetivos Específicos

- **Problema A (Control Térmico):** Implementar un algoritmo de regulación de temperatura en bucle cerrado mediante el sensor DHT11, que gobierne un calefactor (GPIO) y un ventilador de velocidad variable (Fast PWM en Timer2) en función de un Setpoint dinámico reconfigurable por UART, transmitiendo la telemetría del sistema hacia MATLAB.
- **Problema B (Control de Iluminación y Posición):** Desarrollar un sistema de posicionamiento de persiana automatizada impulsado por un motor paso a paso y driver L293D, utilizando realimentación de luz mediante una fotorresistencia (LDR), lectura de posición con potenciómetro en el ADC y evaluación del error con ventana de histéresis.
- **Problema D (Sintetizador de Audio Polifónico):** Construir un piano electrónico capaz de sintetizar dos tonos simultáneos (melodía y sub-octava de bajo) mediante los Timers 1 y 2 en modo CTC, permitiendo la ejecución manual mediante un teclado de pulsadores y la reproducción automática de piezas musicales decodificadas por comandos serie UART.

III. Materiales

Para el desarrollo de las diferentes actividades del laboratorio se utilizaron los siguientes componentes:

- Microcontrolador ATmega328P.
- Placa Arduino Uno (Lafvin).
- Sensor de temperatura (DHT11).
- Fotorresistencia (LDR).
- Tira de LEDs WS2812.
- Servomotor y motor DC con potenciómetro.
- Pantalla LCD 16x2.
- Protoboard, LEDs, resistencias, pulsadores y cables de conexión.

.

III-A. Arduino Uno

Placa de desarrollo basada en microcontrolador que permite programar y controlar circuitos electrónicos mediante entradas y salidas digitales y analógicas.



Figura 1. Arduino Uno.

III-B. ATmega328P

Microcontrolador de 8 bits de la familia AVR. Componente encargado de ejecutar las instrucciones del programa y controlar los periféricos del sistema.

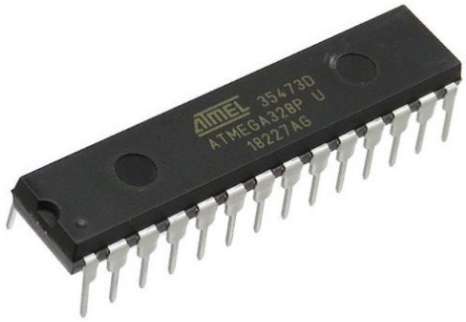


Figura 2. Microcontrolador ATmega328P.

.

III-C. Pantalla LCD 16x2

display de cristal líquido que muestra hasta 32 caracteres alfanuméricos distribuidos en 2 filas de 16 caracteres cada una



Figura 3. Pantalla LCD 16x2.

III-D. Sensor de Temperatura DHT11

sensor digital de bajo costo que mide temperatura y humedad relativa

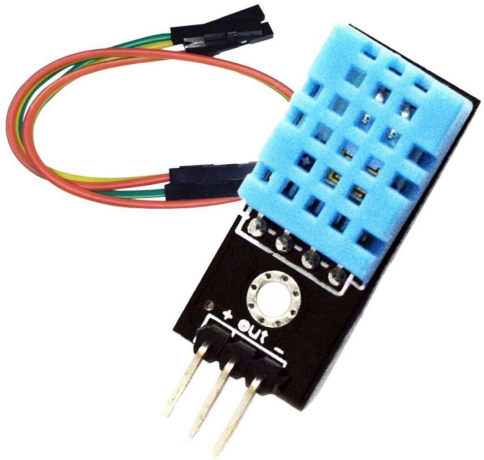


Figura 4. Sensor de temperatura.

.

III-E. Fotorresistencia (LDR)

es un componente electrónico cuya resistencia eléctrica cambia según la cantidad de luz que incide sobre su superficie

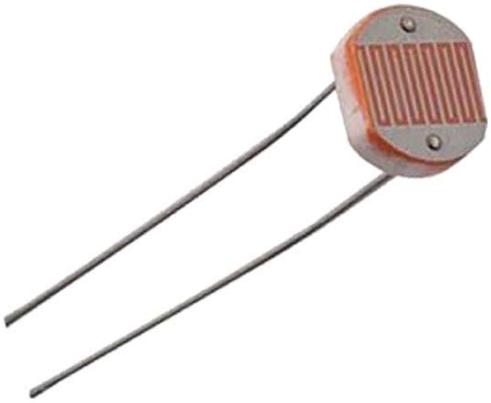


Figura 5. Fotorresistencia

III-F. Tira de LEDs WS2812

cinta flexible de iluminación RGB donde cada LED es direccionable de forma individual



Figura 6. Tira de LEDs WS2812.

.

III-G. Motor DC

una máquina que transforma la energía eléctrica en energía mecánica mediante un movimiento rotatorio generado por campos magnéticos



Figura 7. Motor DC

III-H. Servomotor

motor eléctrico que permite controlar con alta precisión la posición angular, la velocidad y la aceleración de su eje



Figura 8. Servomotor.

.

III-I. Potenciómetro

un componente electrónico de resistencia variable que permite controlar la intensidad de la corriente eléctrica o regular el voltaje en un circuito



Figura 9. potenciómetro.

IV. Problema A: Sistema de Control de Temperatura en Tiempo Real

IV-A. Objetivo

Diseñar e implementar un sistema embebido de control térmico en tiempo real basado en el microcontrolador ATmega328P. El sistema adquiere la temperatura con un sensor DHT11, controla un calefactor (GPIO ON/OFF) y un ventilador (PWM proporcional), despliega el estado en un LCD 16×2 vía I2C, permite reconfigurar el Setpoint (PM) por UART y transmite telemetría en tiempo real a MATLAB.

IV-B. Metodología

1. **Adquisición (1-Wire):** Lectura del sensor DHT11 en el pin `PD2` cada 5 segundos con verificación de checksum.
2. **Setpoint Adaptativo:** PM inicial en 20°C (ajustable vía UART entre 10°C y 35°C). La variación $\Delta = PM - 20$ recalibra automáticamente los umbrales de control.
3. **Control de Actuadores:** Calefactor en pin digital `PD4` y ventilador mediante Fast PWM en Timer2 (`PD3`/`OC2B`).
4. **Despliegue y Telemetría:** Visualización local en LCD I2C y envío serie (9600 baudios, CSV) hacia MATLAB.

.

IV-C. Máquina de Estados

El control se modela mediante 6 estados condicionados por $Temp$ y el desfase $\Delta = PM - 20$:

- **S0 (CALEFACTOR ON):** $Temp \leq 15 + \Delta \implies$ Calefactor ON (`PD4=1`), PWM 0 %.
- **S1 (IDEAL / Standby):** $16+\Delta \leq Temp \leq 25+\Delta \implies$ Ambos actuadores OFF.
- **S2 (FAN_LOW):** $26 + \Delta \leq Temp \leq 35 + \Delta \implies$ PWM al 33 % (OCR2B $= 84$).
- **S3 (FAN_MED):** $Temp > 35 + \Delta \implies$ PWM al 66 % (OCR2B $= 168$).
- **S4 (FAN_HIGH):** $Temp > 45 + \Delta \implies$ PWM al 100 % (OCR2B $= 255$).
- **S5 (ERROR_DHT):** Fallo de lectura del sensor. Salidas desactivadas.

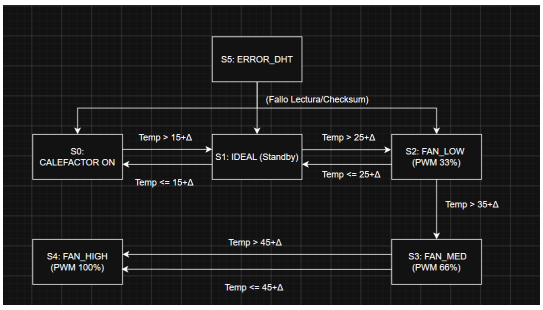


Figura 10. Diagrama de la Máquina de Estados.

IV-D. Implementación Técnica

- **Timer2 (Fast PWM):** Control de velocidad del ventilador en `PD3` con prescaler 1024.
- **Módulo I2C (100 kHz):** Manejo del LCD mediante el integrado PCF8574 en pines `PC4` (`SDA`) y `PC5` (`SCL`).
- **Interrupción UART (`USART_RX_vect`):** Recepción asíncrona de comandos para actualizar PM en segundo plano.

Cuadro I
Mapeo de Pines del ATmega328P

Pin ATmega328P	Etiqueta	Función / Periférico
PD2	DHT_PIN	Entrada Digital (1-Wire DHT11)
PD3	FAN_PIN	Salida PWM Timer2 (OC2B) / LED Verde
PD4	HEATER_PIN	Salida Digital GPIO / LED Rojo
PC4	SDA	Bus I2C TWI (PCF8574 – LCD)
PC5	SCL	Bus I2C TWI (PCF8574 – LCD)
PD0	RXD	Recepción Serie USART
PD1	TXD	Transmisión Serie USART

IV-E. Explicación del Código

El programa principal se divide en cuatro partes clave:

- **Pantalla LCD por I2C:** Inicializa la comunicación I2C a 100 kHz y envía los datos al LCD en bloques de 4 bits mediante el chip PCF8574, usando solo dos pines del microcontrolador.
- **Lectura del DHT11:** Envía la señal de inicio por el pin `PD2`, mide la duración de los 40 bits que devuelve

el sensor y verifica la suma de control (checksum) para obtener la temperatura.

- **Control de Actuadores:** Configura el Timer2 en Fast PWM para ajustar la velocidad del ventilador en `PD3` (0 %, 33 %, 66 % y 100 %) y maneja el encendido directo del calefactor en el pin `PD4`.
- **UART y Bucle Principal:** Recibe nuevos valores de Punto Medio mediante la interrupción `USART_RX_vect`. Cada 5 segundos recalcula los umbrales térmicos y manda los datos a MATLAB en formato CSV.

IV-F. Montaje Físico y Conexionado

El circuito en protoboard interconecta el ATmega328P con el sensor DHT11 (`PD2`), el LCD I2C (`PC4`/`PC5`), y los LEDs de indicación para calefactor (`PD4`, Rojo) y ventilador (`PD3`, Verde).

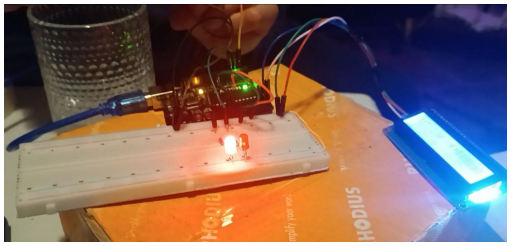


Figura 11. Montaje físico del circuito en protoboard.

IV-G. Simulación y Resultados

La simulación en Proteus VSM se vinculó a MATLAB mediante Virtual Serial Port Kit (`COM3` ↔ `COM4` a 9600 baudios). Se evaluó la respuesta para $PM = 20°\text{C}$ (Zona Ideal: $16°\text{C} - 25°\text{C}$):

IV-G1. Prueba a 9°C (Calefactor ON): A 9°C ($Temp \leq 15°\text{C}$), se activa el calefactor (`PD4=HIGH`, LED Rojo). La LCD despliega `Temp: 9 C` y MATLAB grafica la lectura inicial por debajo del PM.

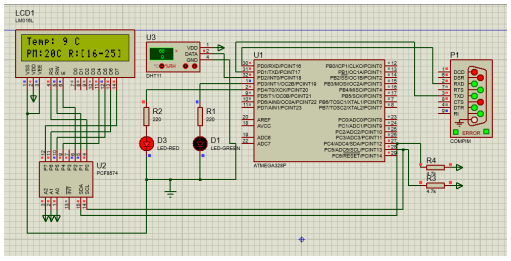


Figura 12. Proteus a 9°C con Calefactor ON.

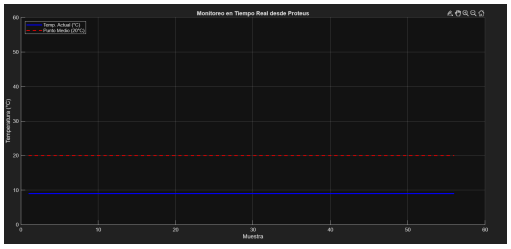


Figura 13. Telemetría en MATLAB a 9°C.

IV-G2. Prueba a 18°C (Zona Ideal / Standby): En confort (18°C), el calefactor se apaga y el ventilador permanece al 0 % PWM (LEDs apagados). MATLAB registra la transición a 18°C.

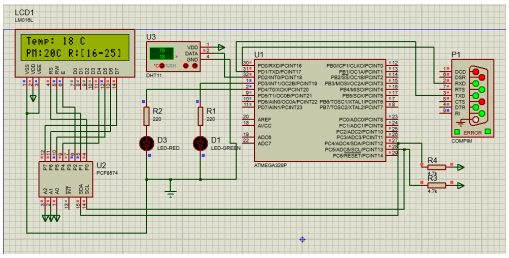


Figura 14. Proteus a 18°C en Zona Ideal.

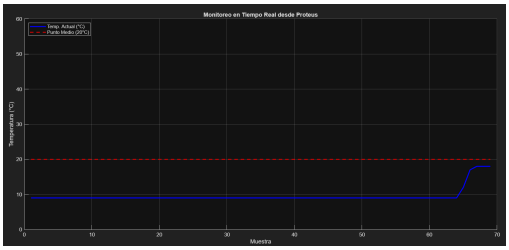


Figura 15. Telemetría en MATLAB a 18°C.

IV-G3. Prueba a 26°C (Ventilador BAJO – 33 % PWM): A 26°C, el Timer2 genera 33 % PWM en `PD3` (OCR2B = 84), encendiendo el LED Verde. MATLAB muestra la curva superando el PM.

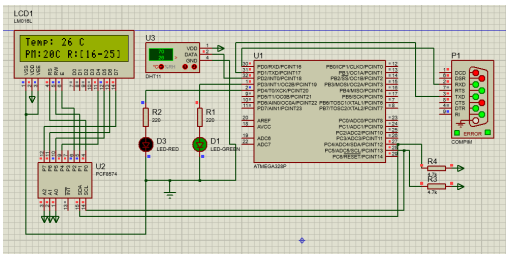


Figura 16. Proteus a 26°C con Ventilador al 33 % PWM.

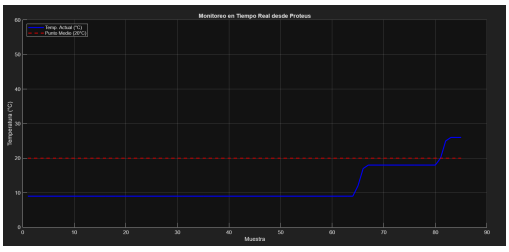


Figura 17. Telemetría en MATLAB a 26°C.

.

IV-G4. Prueba a 36°C (Ventilador MEDIO – 66 % PWM): A 36°C, el PWM aumenta al 66 % (OCR2B = 168), incrementando el brillo del LED Verde. MATLAB registra el tercer escalón ascendente.

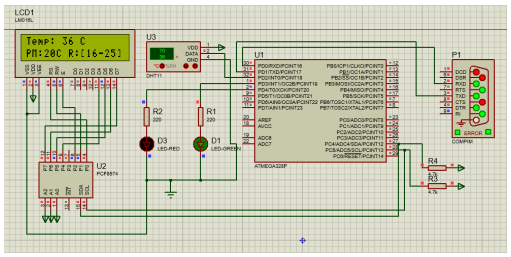


Figura 18. Proteus a 36°C con Ventilador al 66 % PWM.

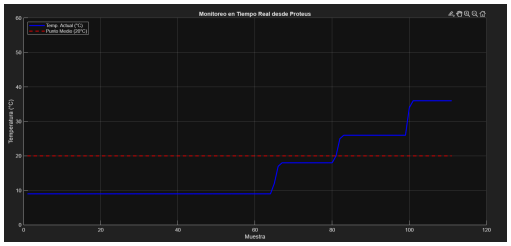


Figura 19. Telemetría en MATLAB a 36°C.

IV-G5. Prueba a 46°C (Ventilador ALTO – 100 % PWM): En temperatura crítica (46°C), el ventilador opera al 100 % PWM (OCR2B = 255). La gráfica consolidada en MATLAB muestra la evolución completa del sistema.

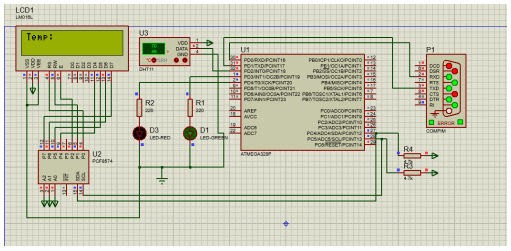


Figura 20. Proteus a 46°C con Ventilador al 100 % PWM.

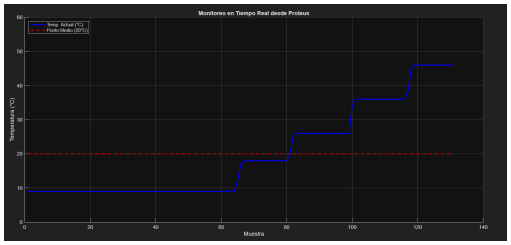


Figura 21. Evolución temporal completa de la telemetría en MATLAB.

IV-H. Conclusiones

El sistema en el ATmega328P cumplió exitosamente con el control térmico y monitoreo en tiempo real. La FSM adaptativa garantizó la regulación precisa entre el calefactor y el ventilador por PWM, mientras que el enlace serie virtual permitió la adquisición continua de telemetría en MATLAB.

V. PROBLEMA B

V-A. Objetivo

Implementar un sistema de control de iluminación utilizando el microcontrolador ATmega328P, que permita establecer un valor de referencia mediante comunicación UART, medir la iluminación con una fotorresistencia y controlar el movimiento de un motor paso a paso según la diferencia entre el valor deseado y el valor medido.

Además, se busca transmitir las variables del sistema a una computadora para observar su comportamiento mediante MATLAB.

V-B. Metodología

En primer lugar, se configuraron los periféricos del ATmega328P necesarios para el funcionamiento del sistema. Se utilizaron las entradas del conversor analógico-digital (ADC) para leer la fotorresistencia (LDR) y el potenciómetro, y se configuró la comunicación UART a 9600 baudios.

El usuario puede enviar desde la computadora un valor de referencia o setpoint entre 0 y 1023. Este valor se recibe por UART y se utiliza como referencia para el control de iluminación.

Luego, el programa obtiene las lecturas de los sensores y calcula el error entre el setpoint y la lectura de la LDR:

$$e = SP - LDR \tag{1}$$

Si el error supera las 15 unidades en valor absoluto, el programa selecciona el sentido de giro del motor según el signo del error. Cuando el error se encuentra dentro del margen establecido, el motor permanece detenido.

El movimiento se realiza mediante una secuencia de cuatro pasos, modificando los retardos entre pasos según el valor calculado para la velocidad. Finalmente, los datos del sistema se transmiten por UART para ser procesados y representados gráficamente en MATLAB. .

V-C. *Máquina de Estados*

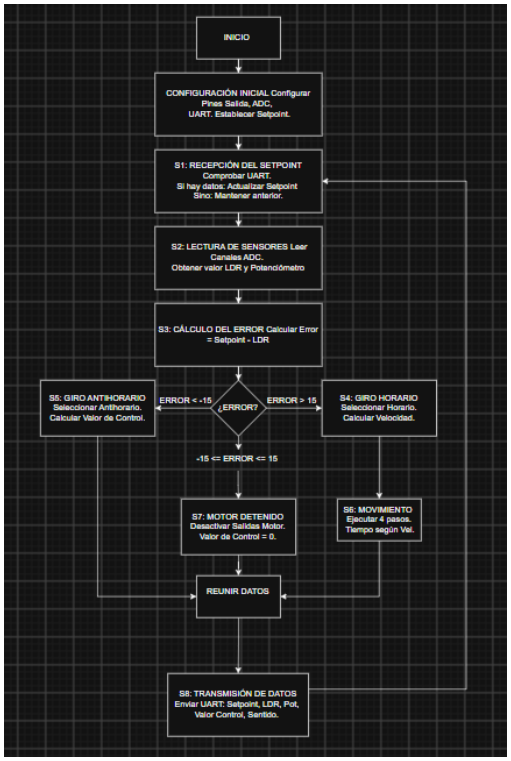


Figura 22. .

V-D. *Descripción de Estados:*

Se configuran los pines de salida utilizados para controlar el motor, el conversor analógico-digital y la comunicación UART. También se establece un valor inicial para el setpoint.

V-D1. S1: Recepción del setpoint: El programa comprueba si hay datos disponibles por UART. Cuando recibe un valor completo dentro del rango permitido, actualiza el setpoint. Si no recibe un nuevo valor, conserva el anterior.

V-D2. S2: Lectura de sensores: Se leen los canales ADC correspondientes a la LDR y al potenciómetro. La lectura de la LDR se utiliza para decidir el movimiento, mientras que el valor del potenciómetro se transmite para supervisar la posición del mecanismo.

V-D3. S3: Cálculo del error: Se calcula la diferencia entre el setpoint y la lectura de la LDR. El resultado permite determinar si la iluminación se encuentra dentro del margen permitido o si es necesario realizar una corrección.

V-D4. S4: Giro horario: Este estado se selecciona cuando el error es mayor que 15 unidades. El programa selecciona el sentido horario y calcula el valor de control utilizado para ajustar la velocidad de movimiento.

V-D5. S5: Giro antihorario: Este estado se selecciona cuando el error es menor que -15. El programa selecciona el sentido antihorario y calcula el valor de control correspondiente.

V-D6. S6: Movimiento del motor: Se ejecutan cuatro pasos consecutivos del motor en el sentido seleccionado. El tiempo de espera entre pasos depende del valor calculado para la velocidad.

V-D7. S7: Motor detenido: Cuando el error se encuentra entre -15 y 15, inclusive, no se ejecuta una secuencia de movimiento. Las salidas del motor se desactivan y el valor de control se establece en cero.

V-D8. S8: Transmisión de datos: Se envían por UART las variables principales del sistema: setpoint, lectura de la LDR, lectura del potenciómetro, valor de control y sentido de giro. Después, el programa vuelve al inicio del ciclo principal.

V-E. *Funcionamiento del programa*

El programa se ejecuta continuamente, repitiendo las etapas de recepción de datos, lectura de sensores, cálculo del error y control del motor.

El sentido de giro depende del signo del error, mientras que la magnitud del error determina el valor de control calculado. Si el error se encuentra dentro de la tolerancia, el motor permanece detenido.

V-F. *Visualización mediante MATLAB*

El programa de MATLAB recibe los datos enviados por UART y representa gráficamente las variables del sistema. Esto permite observar la evolución del setpoint, la lectura de la LDR, el potenciómetro y el valor de control.

La gráfica facilita la comparación entre el valor deseado y el medido, así como la observación de los cambios producidos durante el funcionamiento del sistema.



Figura 23. abriendo perciana.

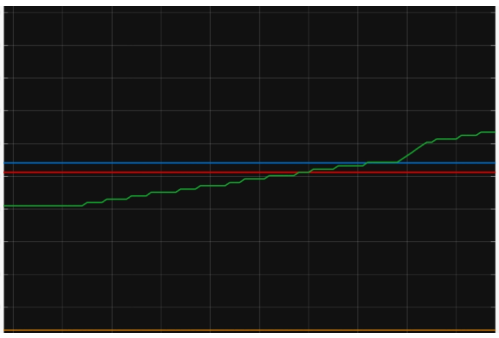


Figura 24. cerrando perciana.

V-G. *Simulación en Proteus*

Para validar el funcionamiento del algoritmo, el comportamiento de las entradas analógicas y la secuencia de conmutación del motor paso a paso, se implementó el circuito esquemático en Proteus 8 Professional

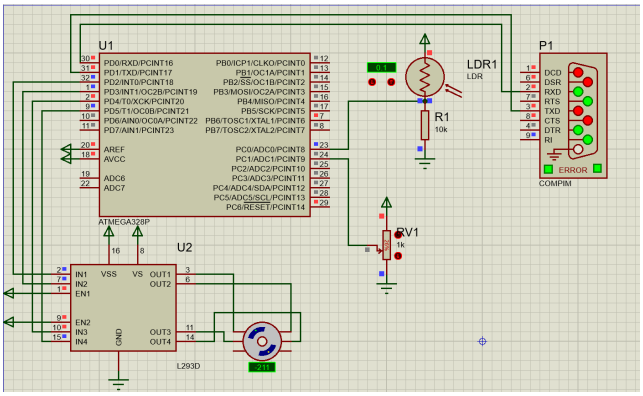


Figura 25. Circuito esquemático de la simulación en Proteus para el Ejercicio B.

V-G1. *Descripción de los Bloques del Circuito:*

- **Microcontrolador ATmega328P ($U1$):** Configurado para operar a una frecuencia de reloj interna/externa de 16 MHz.

- **Sensor de Iluminación ($LDR1$ y $R1$):** Se implementó un divisor de tensión formado por la fotorresistencia $LDR1$ conectada a V_{CC} y una resistencia fija $R_1 = 10\ \text{k}\Omega$ conectada a GND. El punto medio está conectado al canal `ADC0` (pin 23). Al incrementarse la intensidad luminosa, la resistencia de $LDR1$ cae, aumentando la tensión leída por el ADC.

- **Sensor de Posición del Mecanismo ($RV1$):** Un potenciómetro lineal de $1\ \text{k}\Omega$ alimentado entre 5 V y GND entrega su cursor al canal `ADC1` (pin 24), simulando la realimentación de la posición angular de la persiana.

- **Driver de Potencia ($U2$ - L293D):** Se utilizó el circuito integrado L293D (puente H doble) para alimentar las cuatro fases del motor paso a paso. Las entradas de control `IN1`, `IN2`, `IN3` e `IN4` se conectaron a los pines digitales `PD2`, `PD3`, `PD4` y `PD5` del puerto D. Las terminales de habilitación `EN1` y `EN2` se conectaron a V_{CC} para mantener el integrado activo.

- **Interfaz de Comunicación Serial** Permite conectar la simulación de Proteus con un puerto COM virtual de la computadora a 9600 baudios. Las líneas `RXD` (`PD0`) y `TXD` (`PD1`) del ATmega328P se conectaron de forma cruzada a las terminales `TXD` (pin 3) y `RXD` (pin 2) del bloque `COMPIM`, permitiendo recibir el *setpoint* desde una terminal serie y transmitir la telemetría en tiempo real hacia MATLAB.

V-H. *Armado Físico*

Posteriormente a la simulación, se realizó el montaje físico del sistema sobre protoboard utilizando componentes discretos.

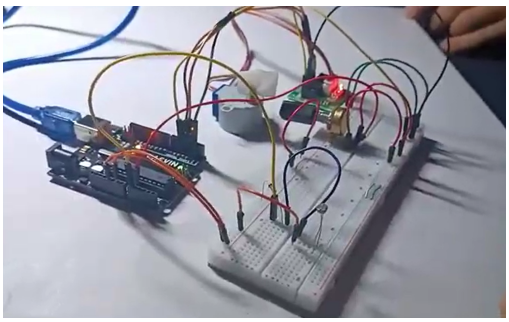


Figura 26. armado fisico.

V-H1. Mapeo de Conexiones Físicas: En la Tabla II se resumen las conexiones físicas realizadas en el microcontrolador ATmega328P.

Cuadro II
ASIGNACIÓN DE PINES DEL ATMEGA328P EN EL PROTOTIPO FÍSICO.

Pin del ATmega328P	Función	Conexión Hardware
PD0 (RXD)	Entrada UART	Transmisión (TX) módulo USB-UART
PD1 (TXD)	Salida UART	Recepción (RX) módulo USB-UART
PD2–PD5	Salidas digitales	Entradas IN1–IN4 del driver L293D
PC0 (ADC0)	Entrada analógica	Divisor de tensión $LDR1/R_1$ (10 kΩ)
PC1 (ADC1)	Entrada analógica	Terminal central del potenciómetro $RV1$
VCC / GND	Alimentación	Fuente regulada de 5 VDC

V-I. *Conclusiones*

En este ejercicio se desarrolló un sistema de control de iluminación utilizando el ATmega328P, la comunicación UART, el conversor analógico-digital y un motor paso a paso.

El programa permite establecer un valor de referencia remoto, medir la iluminación y seleccionar el sentido de giro del motor según el error calculado. Además, la transmisión de datos permite supervisar el funcionamiento mediante MATLAB.

.

VI. Problema D: Piano Electrónico con ATmega328P y Polifonía Dual

VI-A. Objetivo

Diseñar e implementar un piano electrónico embebido basado en el microcontrolador ATmega328P capaz de operar en modo manual mediante un teclado de pulsadores y en modo automático mediante reproducción de canciones predefinidas activadas por comandos UART. El sistema genera polifonía simple (melodía y bajo de acompañamiento) utilizando dos buzzers pasivos y despliega el estado y frecuencias en una pantalla LCD 16×2 vía I2C.

VI-B. Metodología

1. **Polifonía mediante Timers Hardware:** Uso del Timer1 (modo CTC en `PB1`/`OC1A`) para la frecuencia principal de melodía y del Timer2 (modo CTC con prescaler 256 en `PB3`/`OC2A`) para la frecuencia de bajo/acompañamiento.
2. **Etapa de Potencia para Buzzer Secundario:** La señal del bajo en `PB3` conmuta la base de un transistor NPN 2N2222 ($R_1 = 10$ kΩ) para amplificar la corriente del segundo buzzer (`LS2`).
3. **Lectura de Teclado Manual:** Configuración de pines de entrada con resistencias *pull-up* internas en los puertos B, C y D para la lectura de los pulsadores en lógica negativa.
4. **Control por UART:** Interrupción de recepción (`USART_RX_vect`) a 9600 baudios que procesa comandos (`C1`, `C2`, `C3` para reproducción de temas predefinidos y `STOP` para retornar al modo piano).
5. **Interfaz de Usuario (LCD I2C):** Visualización del modo activo, nota/canción en ejecución y frecuencias de melodía y bajo mediante el protocolo TWI/I2C (100 kHz).

VI-C. Máquina de Estados Finitos (FSM)

El control principal del sistema commuta entre los siguientes estados:

- **S0 (MODO PIANO / IDLE):** Estado de espera. Monitorea los pulsadores; si no hay tecla presionada, apaga las salidas de audio.
- **S1 (MODO PIANO / PLAYING):** Al presionar una tecla, genera la frecuencia fundamental en Timer1 y la sub-octava de bajo en Timer2, mostrando la nota y frecuencia en la LCD.
- **S2 (MODO CANCIÓN / REPRODUCCIÓN):** Se activa al recibir comandos UART (`C1`, `C2` o `C3`). Secuencia los eventos de la melodía y bajo almacenados en Flash. Se interrumpe inmediatamente si se recibe el comando `STOP`.

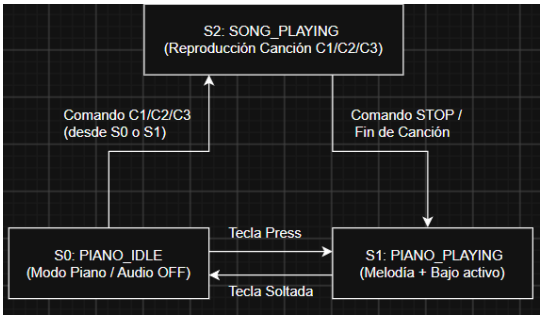


Figura 27. Diagrama de la Máquina de Estados Finitos (FSM).

VI-D. Implementación Técnica

- **Timer1 (Melodía):** Configurado en modo CTC (`WGM12`) alternando el pin `OC1A` (`PB1`) según el cálculo: $\text{OCR1A} = \frac{F_{CPU}}{2 \cdot 8 \cdot f} - 1$.
- **Timer2 (Bajo):** Configurado en modo CTC (`WGM21`) sobre `OC2A` (`PB3`) con prescaler 256 para permitir frecuencias graves sin desbordamiento de 8 bits.
- **Lógica de Recepción Serie:** Buffer circular de entrada que valida cadenas finalizadas en retorno de carro o comandos directos (`C1`, `C2`, `C3`, `STOP`).

Cuadro III
Mapeo de Pines del ATmega328P

Pin ATmega328P	Etiqueta	Función / Periférico
PB1	OC1A	Salida PWM Melodía (Buzzer LS1)
PB3	OC2A	Salida PWM Bajo (Base Transistor 2N2222 / LS2)
PC0, PC1	KEY1--KEY2	Entradas Digitales Teclas La3, Si3
PD2–PD7	KEY3--KEY8	Entradas Digitales Teclas Do4, Re4, Mi4, Fa4, Sol4, La4
PB0, PB2	KEY9--KEY10	Entradas Digitales Teclas Si4, Do5
PC4	SDA	Bus I2C TWI (PCF8574 – LCD)
PC5	SCL	Bus I2C TWI (PCF8574 – LCD)
PD0	RXD	Recepción Serie USART (Terminal Virtual)
PD1	TXD	Transmisión Serie USART

VI-E. Explicación del Código

El programa del piano electrónico se organiza en los siguientes módulos principales:

- **Control del LCD vía I2C:** Configura el módulo TWI a 100 kHz para la pantalla en la dirección `0x27` (PCF8574), enviando las instrucciones divididas en nibles de 4 bits.
- **Audio Polifónico con Timers:** Utiliza el Timer1 en modo CTC en el pin `PB1` para la melodía principal y el Timer2 en modo CTC en `PB3` (con prescaler 256) para el bajo de acompañamiento conectado al transistor 2N2222.
- **Lectura del Teclado:** Escanea los 10 pulsadores en lógica negativa. Al detectar una tecla presionada, genera la frecuencia fundamental en la melodía y su sub-octava grave en el bajo.
- **Control UART y Canciones:** La interrupción `USART_RX_vect` procesa los comandos serie (`C1`, `C2`, `C3` y `STOP`). Durante la reproducción automática, la función `custom_delay_ms` evalúa el puerto serie cada 10 ms para permitir la interrupción inmediata del tema.

VI-F. Montaje Físico y Conexionado

El circuito físico interconecta el microcontrolador ATmega328P con el teclado de pulsadores conectados a GND, el expansor LCD I2C en los pines `PC4`/`PC5`, el buzzer principal en `PB1` y el buzzer secundario acoplado al colector del transistor 2N2222 impulsado desde `PB3`.

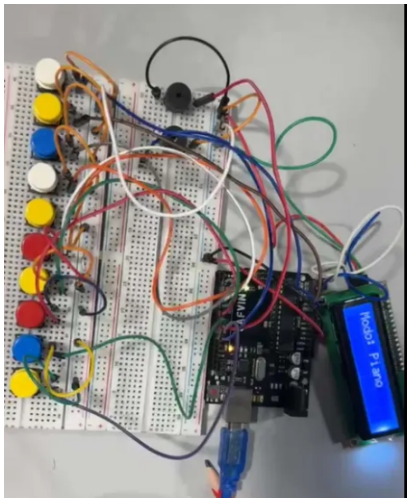


Figura 28. Montaje físico del circuito en protoboard.

VI-G. Simulación y Resultados

La simulación en Proteus VSM validó la operación en ambos modos. Desde la Terminal Virtual se enviaron los comandos serie `C1`, `C2` y `C3`, verificando en el LCD la actualización del título del tema y las frecuencias instantáneas generadas (M : Hz, B : Hz). Asimismo, se comprobó la conmutación inmediata al modo manual ante el comando `STOP` o la pulsación individual de las teclas.

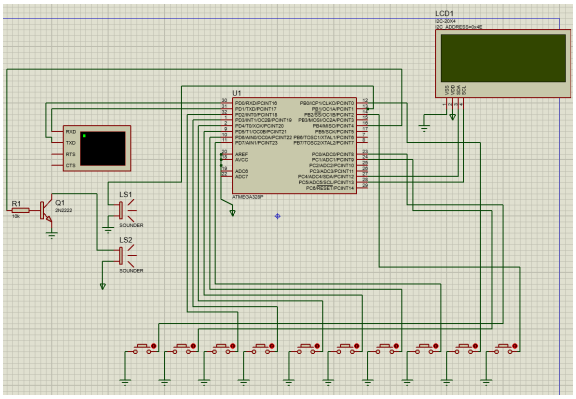


Figura 29. Simulación del sistema en Proteus con Terminal Virtual, LCD I2C y etapa polifónica con 2N2222.

VI-H. Conclusiones

La implementación del piano electrónico demostró la versatilidad de los Timers del ATmega328P para la generación polifónica en tiempo real. La integración del transistor NPN permitió aislar la carga del segundo buzzer asegurando una intensidad acústica equilibrada. Por último, la gestión asíncrona por UART facilitó el control remoto del sistema sin interferir con la reproducción temporal exacta de las canciones.

VII. Bibliografía

- **ATmega328P Datasheet (Microchip / Atmel):** https://www.alldatasheet.com/datasheet-pdf/pdf/241077/ATMEL/ATMEGA328P.html
- **DHT11 Temperature & Humidity Sensor Datasheet:** https://www.alldatasheet.com/datasheet-pdf/pdf/2193416/OSEPP/DHT11.html
- **Micro Servomotor SG90 Datasheet:** https://www.alldatasheet.es/datasheet-pdf/pdf/1572383/ETC/SG90.html
- **Pantalla LCD 16x2 / Adaptador I2C PCF8574 Datasheet:** https://www.alldatasheet.com/datasheet-pdf/pdf/2215051/DFROBOT/LCD1602.html
- **Módulo Buzzer Pasivo OKY3191 Datasheet:** https://agelectronica.lat/pdfs/textos/O/OKY3191.PDF
- **Otras Fuentes:** Documentos, guías prácticas de laboratorio y presentaciones del curso de Microcontroladores.

.

VIII. Link Github y Videos de Drive

- Link Github: https://github.com/brunobentancourt/laboratorio3

- Link Drive: https://drive.google.com/drive/folders/1Ih96zSwqPUDb16xgUtmX0nlbej9bg7bc?hl=es